Learning from Nature: Nature-Inspired Metaheuristics for Machine Learning

Percival Phakathi

"Go to the ant, O sluggard; consider her ways, and be wise."
— Proverbs 6:6

Abstract

Many of the optimisation problems that arise when building machine learning (ML) systems cannot be solved by gradient descent. Examples include tuning hyperparameters, selecting features, searching neural architectures and choosing ensemble members. In these problems the objective is often non-differentiable, noisy, discrete or expensive to evaluate. Nature-inspired metaheuristics are population-based, derivative-free search methods modelled on the collective behaviour of animals, and they offer a practical alternative. This article reviews five of them from an ML perspective: the Firefly Algorithm (FA), the Bat Algorithm (BA), Ant Colony Optimisation (ACO), Particle Swarm Optimisation (PSO) and the Shuffled Frog Leaping Algorithm (SFLA). For each algorithm we:

- state its mathematical formulation and define every symbol
- give its binary or discrete variant where one is needed for ML tasks
- survey its applications across the ML pipeline
- show how to import a Python implementation

We then propose an experimental protocol for comparing metaheuristics fairly against standard ML baselines such as random search and Bayesian optimisation. We close with the limitations of the approach.

Keywords: metaheuristics, swarm intelligence, hyperparameter optimisation, feature selection, neural architecture search, derivative-free optimisation

1 Introduction

Training a model by gradient descent solves only one of the optimisation problems in an ML project. Around it sit several *outer* problems:

- **Which hyperparameters?** For example, the regularisation strength C and kernel width γ of a support vector machine (SVM), or the learning rate and depth of a gradient-boosted ensemble.
- **Which features?** Choosing a subset of d features is a combinatorial problem with 2^d candidates.
- **Which architecture?** The number of layers, the layer types and the connections between them.
- **Which pipeline?** The ordering of preprocessing, feature engineering and model steps.

These problems share four properties that rule out gradient-based methods:

1. The objective (usually cross-validated error) has no useful gradient.

2. The search space is often mixed: continuous, integer and categorical variables together.
3. Every evaluation requires training a model, so evaluations are expensive.
4. The landscape is frequently *multimodal*: many quite different configurations perform almost equally well.

Nature-inspired metaheuristics are well suited to this setting. They keep a *population* of candidate solutions, which they evaluate only through the objective value. They update the population with stochastic rules modelled on animal behaviour and balance two forces: *exploration* (searching new regions) and *exploitation* (refining promising ones). They do not guarantee the global optimum, but they are flexible, easy to parallelise and need no assumptions about the shape of the objective.

Organisation of the article. Section 2 expresses common ML tasks as optimisation problems and defines a shared Python objective. Sections 3–7 then cover the five algorithms in turn. Each of these sections gives the biological inspiration, the mathematical formulation with every symbol defined, the binary variant, applications in ML and a Python implementation. Each list of applications covers the general uses of the algorithm in the ML pipeline and ends with industry examples showing how an organisation could use it in practice:

- **FA:** tuberculosis screening in healthcare and crop-yield forecasting in agriculture.
- **BA:** day-ahead electricity demand forecasting in energy and store-level sales forecasting in retail.
- **ACO:** explainable fraud rules in insurance and customer churn prediction in telecommunications.
- **PSO:** card-fraud detection in banking and customer segmentation in retail and marketing.
- **SFLA:** explainable credit scoring in banking and predictive maintenance in manufacturing.

Section 8 compares the five algorithms side by side, Section 9 proposes an experimental protocol for fair comparisons, Section 10 discusses limitations and Section 11 concludes.

2 Preliminaries: ML Problems as Optimisation Problems

2.1 General formulation

Every problem in this article takes the form

$$\mathbf{x}^* = \arg\min_{\mathbf{x}\in\mathcal{X}} f(\mathbf{x}). \tag{1}$$

Symbol	Definition
$\mathbf{x}$	A candidate solution (a vector of decision variables)
$\mathcal{X}$	The search space (box constraints, binary vectors or permutations)
$f(\mathbf{x})$	The objective, or *fitness*, function to minimise
$\mathbf{x}^*$	The best solution

Maximisation problems are handled by minimising $-f$. Throughout, N denotes the population size, D the dimension of $\mathbf{x}$ and t the iteration counter.

2.2 Hyperparameter optimisation (HPO)

$$\boldsymbol{\lambda}^* = \arg\min_{\boldsymbol{\lambda} \in \Lambda} \; \frac{1}{K} \sum_{k=1}^{K} \mathcal{L}\Big(\mathcal{A}_{\boldsymbol{\lambda}}(\mathcal{D}_{\text{train}}^{(k)}),\; \mathcal{D}_{\text{val}}^{(k)}\Big) \tag{2}$$

Symbol	Definition
$\boldsymbol{\lambda}$	A hyperparameter configuration
Λ	The hyperparameter search space
$\mathcal{A}_{\boldsymbol{\lambda}}$	The learning algorithm run with hyperparameters $\boldsymbol{\lambda}$
$\mathcal{D}_{\text{train}}^{(k)}, \mathcal{D}_{\text{val}}^{(k)}$	The training and validation sets of fold k
K	The number of cross-validation folds
$\mathcal{L}$	The validation loss, for example $1 -$ accuracy or RMSE

Scale-type hyperparameters are searched on a logarithmic scale, $\lambda = 10^x$ with $x \in [x_{\min}, x_{\max}]$. Integer hyperparameters are obtained by rounding, $\lambda = \lfloor x \rceil$.

2.3 Feature selection

A feature subset is encoded as a binary mask $\mathbf{s} \in \{0,1\}^d$, where $s_j = 1$ keeps feature j. A common fitness function trades error against subset size:

$$F(\mathbf{s}) = \alpha\, \varepsilon(\mathbf{s}) + (1 - \alpha)\, \frac{\|\mathbf{s}\|_1}{d}. \tag{3}$$

Symbol	Definition
$\varepsilon(\mathbf{s})$	The cross-validated error of a classifier trained on the selected features
$\|\mathbf{s}\|_1$	The number of selected features
d	The total number of features
$\alpha \in [0,1]$	The weight on error against compactness (typically 0.9–0.99)

2.4 From continuous to binary: transfer functions

FA, BA, PSO and SFLA work in continuous space. For binary problems such as feature selection, each continuous component v_j (a position or velocity) is mapped to a probability with a *transfer function*, and the bit is then sampled (Mirjalili and Lewis, 2013):

$$T_S(v_j) = \frac{1}{1 + e^{-v_j}}, \qquad s_j = \begin{cases} 1 & \text{if } r < T_S(v_j), \\ 0 & \text{otherwise,} \end{cases} \qquad r \sim U(0,1). \tag{4}$$

T_S is the *S-shaped* (sigmoid) transfer function. A *V-shaped* alternative, $T_V(v_j) = |\tanh(v_j)|$, flips the current bit with probability $T_V(v_j)$ instead of setting it.

2.5 Clustering and weight training

In centroid-based clustering, a candidate solution concatenates K centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$ and minimises the within-cluster sum of squares:

$$J(\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K) = \sum_{i=1}^{n} \min_{k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2. \tag{5}$$

For neural network training without gradients, the candidate solution is the flattened weight vector $\mathbf{w}$:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \; \frac{1}{n} \sum_{i=1}^{n} \ell(y_i,\, g(\mathbf{x}_i; \mathbf{w})). \tag{6}$$

Here g is the network, ℓ is the per-sample loss and n is the number of training samples. This is practical only for small networks, because D grows with the number of weights.

2.6 A shared objective function

All five algorithms below can optimise the same Python objective. The example tunes an SVM on the Wisconsin breast cancer dataset, searching $\log_{10} C$ and $\log_{10} \gamma$:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_breast_cancer(return_X_y=True)

def objective(sol):
    """sol = [log10(C), log10(gamma)]; returns the 5-fold CV error to minimise."""
    C, gamma = 10 ** sol[0], 10 ** sol[1]
    model = make_pipeline(StandardScaler(), SVC(C=C, gamma=gamma))
    return 1 - cross_val_score(model, X, y, cv=5).mean()
```

With Mealpy (version 3.x), any of its optimisers is attached to this objective in a few lines:

```
from mealpy import FloatVar, PSO

problem = {"obj_func": objective, "bounds": FloatVar(lb=(-3, -5), ub=(3, 1)), "
    minmax": "min"}
best = PSO.OriginalPSO(epoch=50, pop_size=20).solve(problem)
print(best.solution, best.target.fitness)
```

3 Firefly Algorithm (FA)

3.1 Biological inspiration

Fireflies (family *Lampyridae*) produce rhythmic flashes of light to attract mates. A brighter flash attracts more mates, but light intensity falls with distance and is absorbed by the air, so attraction is local. Yang (2008, 2009) abstracted this into an optimiser in which brightness encodes solution quality.

3.2 Mathematical formulation

The brightness of firefly i is its fitness, $I_i \propto -f(\mathbf{x}_i)$ for minimisation. Attractiveness decays with distance:

$$\beta(r_{ij}) = \beta_0 \, e^{-\gamma r_{ij}^2}, \qquad r_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|_2. \tag{7}$$

If firefly j is brighter than firefly i ($I_j > I_i$), firefly i moves toward it:

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \beta_0 \, e^{-\gamma r_{ij}^2} \left(\mathbf{x}_j^t - \mathbf{x}_i^t\right) + \alpha_t \, \boldsymbol{\epsilon}_i^t. \tag{8}$$

The random step is usually reduced over time:

$$\alpha_t = \alpha_0 \, \theta^t, \qquad 0 < \theta < 1. \tag{9}$$

Symbol	Definition
$\mathbf{x}_i^t$	Position (candidate solution) of firefly i at iteration t
I_i	Brightness of firefly i, determined by its fitness
r_{ij}	Euclidean distance between fireflies i and j
β_0	Attractiveness at distance $r = 0$ (commonly 1)
γ	Light-absorption coefficient: controls how fast attraction fades with distance
α_t	Randomisation (step-size) parameter at iteration t
α_0	Initial randomisation parameter
θ	Decay rate of the randomisation parameter
$\boldsymbol{\epsilon}_i^t$	Random vector drawn from $U(-0.5, 0.5)^D$ or $\mathcal{N}(\mathbf{0}, \mathbf{I})$

Two limiting cases explain the behaviour. As $\gamma \to 0$, attraction no longer fades with distance, and FA behaves like a fully connected swarm that converges on a single optimum. As $\gamma \to \infty$, attraction disappears, and FA becomes a random search. Intermediate values make the population split into sub-swarms, each settling on a different local optimum. This is why FA suits *multimodal* problems.

3.3 Binary variant

For feature selection, the continuous update above produces a displacement Δx_{ij} for each dimension j. That displacement is passed through the transfer function of Section 2.4 to obtain the bit s_{ij}.

3.4 Applications in machine learning

1. **SVM hyperparameter tuning.** Search $(\log_{10} C, \log_{10} \gamma)$ to minimise cross-validated error. Several regions of this space often perform similarly, and FA can find more than one of them.

2. **Gradient-boosting tuning.** Search learning rate, maximum depth, subsample ratio and number of estimators for XGBoost or LightGBM. Integers are obtained by rounding.

3. **Wrapper feature selection.** Use binary FA with the fitness function $F(\mathbf{s})$ of Section 2.3, for example on gene-expression data, where d is in the thousands.

4. **Clustering.** Each firefly encodes K centroids and minimises J of Section 2.5. Alternatively, FA can provide initial centroids for k-means and reduce its sensitivity to initialisation.

5. **Training small neural networks.** Optimise the weight vector $\mathbf{w}$ of a shallow multilayer perceptron, which is useful when the loss is non-differentiable (for example, directly optimising F1).

6. **Diverse ensembles.** Because FA's sub-swarms settle on different optima, the best firefly of each sub-swarm can serve as a base learner. This gives an ensemble of accurate but diverse models.

7. **Image preprocessing.** Choose multilevel thresholds that maximise Otsu's between-class variance (Otsu, 1979) before segmentation or classification.

8. **Healthcare: tuberculosis screening from chest X-rays.** A screening programme could use FA to tune several image classifiers at once, keeping the best model from each sub-swarm. Where these diverse models disagree on an X-ray, the case is sent to a radiologist, so specialist time goes to the hardest cases.

9. **Agriculture: crop-yield forecasting.** A farming cooperative could use FA to choose which weather, soil and satellite-vegetation features a yield model uses and to tune its

settings. Because FA returns several good configurations, the cooperative gets a range of forecasts rather than a single number, which helps with planning storage and sales.

3.5 Python

```python
from mealpy import FloatVar, BinaryVar, FFA          # FFA.OriginalFFA (Mealpy 3.x)
from niapy.algorithms.basic import FireflyAlgorithm  # NiaPy alternative
```

4 Bat Algorithm (BA)

4.1 Biological inspiration

Microbats navigate and hunt by *echolocation*: they emit ultrasonic pulses and interpret the returning echoes. While searching, the calls are loud and infrequent. As a bat closes in on prey, its calls become quieter and more frequent. Yang (2010) modelled this transition from broad search to precise pursuit.

4.2 Mathematical formulation

Each bat i has a position $\mathbf{x}_i$, velocity $\mathbf{v}_i$, frequency f_i, loudness A_i and pulse emission rate r_i. At each iteration:

$$f_i = f_{\min} + (f_{\max} - f_{\min})\,\beta, \qquad \beta \sim U(0,1), \tag{10}$$

$$\mathbf{v}_i^{t+1} = \mathbf{v}_i^t + \left(\mathbf{x}_i^t - \mathbf{x}_*\right) f_i, \tag{11}$$

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \mathbf{v}_i^{t+1}. \tag{12}$$

With probability depending on the pulse rate (when rand $> r_i$), the bat also takes a *local random walk* around the current best:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_* + \epsilon\,\bar{A}^t, \qquad \epsilon \sim U(-1,1). \tag{13}$$

The new solution is accepted if it improves the fitness and rand $< A_i$. When it is accepted, loudness and pulse rate are updated:

$$A_i^{t+1} = \alpha\,A_i^t, \qquad r_i^{t+1} = r_i^0\left[1 - e^{-\gamma t}\right]. \tag{14}$$

Symbol	Definition
$\mathbf{x}_i^t, \mathbf{v}_i^t$	Position and velocity of bat i at iteration t
f_i	Pulse frequency of bat i; it scales the step toward the best bat
$f_{\min}, f_{\max}$	Frequency range (for example 0 and 2)
β	A uniform random number in $[0,1]$
$\mathbf{x}_*$	Current global best solution
$\bar{A}^t$	Mean loudness of all bats at iteration t
A_i	Loudness of bat i; high values mean exploration, and it decreases over time
r_i, r_i^0	Pulse emission rate of bat i and its initial value; it increases over time
α	Loudness decay constant, $0 < \alpha < 1$ (commonly 0.9)
γ	Pulse-rate growth constant, $\gamma > 0$ (commonly 0.9)

As $t \to \infty$, $A_i^t \to 0$ and $r_i^t \to r_i^0$. The search shifts automatically from exploration to exploitation, much like a built-in annealing schedule.

4.3 Binary variant

The Binary Bat Algorithm (BBA) of Nakamura et al. (2012) keeps the velocity update above. It replaces the position update with a transfer function that converts each velocity component into a bit-flip probability, as in Section 2.4. BBA was proposed specifically for feature selection.

4.4 Applications in machine learning

1. **Wrapper feature selection.** Use BBA with $F(\mathbf{s})$ to choose a compact feature subset, for example selecting among the 30 morphological features of the Wisconsin breast cancer dataset.
2. **Hyperparameter tuning of deep networks.** Search learning rate (log scale), dropout rate, batch size and number of units. The decaying loudness suits expensive objectives, because later evaluations concentrate around the best configuration.
3. **Medical image segmentation.** Choose multilevel thresholds that maximise Otsu's variance or Kapur's entropy (Kapur et al., 1985), as a preprocessing step for tumour detection.
4. **Training feedforward networks.** Search the weights and biases of small networks directly, avoiding local minima that gradient descent can fall into on small, noisy datasets.
5. **Clustering.** Use encoded centroids as in Section 2.5, with the local random walk refining cluster centres in later iterations.
6. **Class-imbalance handling.** Tune class weights or resampling ratios (for example the SMOTE oversampling ratio) to maximise recall or the area under the precision-recall curve on the minority class.
7. **Time-series forecasting.** Tune the look-back window and hyperparameters of LSTM or gradient-boosting forecasters.
8. **Energy: day-ahead electricity demand forecasting.** A utility such as Eskom could use BA to tune how many past hours and previous days a demand forecaster looks at, together with its model settings. More accurate day-ahead forecasts reduce both the risk of load shedding and the cost of keeping unneeded generation on standby.
9. **Retail: store-level sales forecasting.** A retailer could use BA to tune one forecaster per product category for replenishment, where better forecasts mean fewer stock-outs and less wasted stock.

4.5 Python

```
from mealpy import FloatVar, BinaryVar, BA      # BA.OriginalBA (Mealpy 3.x)
from niapy.algorithms.basic import BatAlgorithm  # NiaPy alternative
```

5 Ant Colony Optimisation (ACO)

5.1 Biological inspiration

Foraging ants deposit pheromone on the ground. Shorter paths between nest and food are travelled more often per unit time, so pheromone accumulates on them faster. Pheromone also evaporates, which gradually erases paths that are no longer used. Through this positive feedback, the colony converges on short paths without central control (Goss et al., 1989). Dorigo (1992) turned this into ACO, originally for combinatorial problems on graphs.

5.2 Mathematical formulation (discrete ACO)

Solutions are built step by step on a construction graph. Ant k at node i moves to node j with probability

$$p_{ij}^{k} = \frac{[\tau_{ij}]^{\alpha}\,[\eta_{ij}]^{\beta}}{\sum_{l\in\mathcal{N}_i^k}[\tau_{il}]^{\alpha}\,[\eta_{il}]^{\beta}}, \qquad j \in \mathcal{N}_i^k. \tag{15}$$

After all m ants have built their solutions, pheromone evaporates and is then reinforced:

$$\tau_{ij} \leftarrow (1-\rho)\,\tau_{ij} + \sum_{k=1}^{m}\Delta\tau_{ij}^{k}, \qquad \Delta\tau_{ij}^{k} = \begin{cases} Q/L_k & \text{if ant } k \text{ used edge } (i,j), \\ 0 & \text{otherwise.} \end{cases} \tag{16}$$

Symbol	Definition
τ_{ij}	Pheromone level on edge (i,j): the colony's learned preference
η_{ij}	Heuristic desirability of edge (i,j), known before the search (for example $1/d_{ij}$ in routing)
α, β	Relative weights of pheromone and heuristic information
$\mathcal{N}_i^k$	Nodes still available to ant k at node i
$\rho \in (0,1]$	Evaporation rate, which prevents premature convergence
m	Number of ants
L_k	Cost of ant k's solution (for example tour length or validation error)
Q	A positive constant scaling the pheromone deposit

5.3 ACO for feature selection

Here the construction graph is the set of features, and an ant adds features one at a time. Feature j is chosen with probability

$$p_j = \frac{\tau_j^{\alpha}\,\eta_j^{\beta}}{\sum_{u\in\mathcal{U}}\tau_u^{\alpha}\,\eta_u^{\beta}}, \qquad \eta_j = I(X_j;Y), \tag{17}$$

where $\mathcal{U}$ is the set of features not yet selected and η_j is a *filter* score, such as the mutual information between feature X_j and the target Y. This is a natural hybrid: the filter score guides the ants, and the wrapper's cross-validated error determines the pheromone deposit.

5.4 ACO for continuous domains ($\text{ACO}_{\mathbb{R}}$)

Socha and Dorigo (2008) replaced the discrete pheromone table with an archive of k good solutions $\mathbf{s}_1, \ldots, \mathbf{s}_k$, sorted by quality. Dimension i of a new solution is sampled from a Gaussian kernel:

$$G^{i}(x) = \sum_{l=1}^{k}\omega_l\,\mathcal{N}\!\left(x;\,\mu_l^i,\,\sigma_l^i\right), \qquad \mu_l^i = s_l^i, \tag{18}$$

$$\omega_l = \frac{1}{qk\sqrt{2\pi}}\,e^{-\frac{(l-1)^2}{2q^2k^2}}, \qquad \sigma_l^i = \xi\sum_{e=1}^{k}\frac{|s_e^i - s_l^i|}{k-1}. \tag{19}$$

Symbol	Definition
k	Size of the solution archive
$\mathbf{s}_l$	The l-th best solution in the archive
ω_l	Weight of solution l; better-ranked solutions are sampled more often
q	Locality parameter: small q favours the best solutions
ξ	Spread parameter, playing the role of the evaporation rate
σ_l^i	Standard deviation of the kernel around solution l in dimension i

$\text{ACO}_{\mathbb{R}}$ makes ACO applicable to continuous problems such as hyperparameter tuning and weight training.

5.5 Applications in machine learning

1. **Feature selection.** Use the graph construction of Section 5.3, guided by mutual information or chi-squared scores.
2. **Classification rule induction.** Ant-Miner (Parpinelli et al., 2002) builds interpretable IF–THEN rules, where each ant constructs a rule term by term. This is a useful approach when explainability matters.
3. **Neural architecture search.** Treat layer choices as nodes in a graph, so that each ant's path is a network architecture. DeepSwarm (Byla and Pang, 2019) applies this idea to convolutional networks.
4. **AutoML pipeline construction.** Each ant chooses a sequence of steps (imputer, then scaler, then feature transformer, then model), and the pheromone learns which sequences validate well.
5. **Bayesian network structure learning.** Ants add directed edges between variables, and the pheromone is reinforced by a scoring function such as the Bayesian information criterion (BIC).
6. **Hyperparameter tuning and weight training.** Use $\text{ACO}_{\mathbb{R}}$ for continuous search spaces.
7. **Clustering.** Assign data points to clusters as a labelling problem, with pheromone recording point–cluster affinities.
8. **Routing and scheduling in ML-driven operations.** For example, a demand model predicts delivery volumes and ACO plans the routes. This is the classic setting for ACO.
9. **Insurance: explainable fraud rules.** An insurer could use Ant-Miner to learn short IF–THEN rules that flag suspicious claims, such as a high-value claim made soon after a policy starts. Investigators can read and check each rule, which is much harder with a black-box model.
10. **Telecommunications: customer churn prediction.** A mobile operator could use ACO, guided by mutual-information scores, to pick a small set from the hundreds of usage, billing and network features it records. A leaner churn model is cheaper to run every day and makes it clearer to the retention team why a customer is at risk.

5.6 Python

```
from sko.ACA import ACA_TSP          # discrete ACO for permutations and routing (
    scikit-opt)
from mealpy import FloatVar, ACOR    # ACOR.OriginalACOR, ACO for continuous domains
     (Mealpy 3.x)
```

6 Particle Swarm Optimisation (PSO)

6.1 Biological inspiration

Bird flocks and fish schools show complex, coordinated movement that emerges from a few local rules: avoid collisions, align with nearby neighbours and stay close to the group (Reynolds, 1987). Kennedy and Eberhart (1995) built on this social behaviour to create PSO. Each particle balances its own experience against the experience of the swarm.

6.2 Mathematical formulation

With the inertia weight introduced by Shi and Eberhart (1998), each particle is updated as follows:

$$\mathbf{v}_i^{t+1} = w\,\mathbf{v}_i^t + c_1\mathbf{r}_1 \odot \left(\mathbf{p}_i - \mathbf{x}_i^t\right) + c_2\mathbf{r}_2 \odot \left(\mathbf{g} - \mathbf{x}_i^t\right), \tag{20}$$

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \mathbf{v}_i^{t+1}. \tag{21}$$

The inertia weight is often decreased linearly:

$$w_t = w_{\max} - (w_{\max} - w_{\min})\,\frac{t}{T}. \tag{22}$$

Symbol	Definition
$\mathbf{x}_i^t, \mathbf{v}_i^t$	Position and velocity of particle i at iteration t
w	Inertia weight: how much of its previous velocity a particle keeps
$w_{\max}, w_{\min}$	Initial and final inertia (commonly 0.9 and 0.4)
T	Maximum number of iterations
c_1	Cognitive coefficient: the pull toward the particle's own best position
c_2	Social coefficient: the pull toward the swarm's best position
$\mathbf{r}_1, \mathbf{r}_2$	Random vectors with components drawn from $U(0,1)$
$\odot$	Element-wise (Hadamard) product
$\mathbf{p}_i$	Personal best position found so far by particle i
$\mathbf{g}$	Global best position found so far by the swarm

Constriction factor. Clerc and Kennedy (2002) proposed an alternative that guarantees convergence for suitable parameters:

$$\mathbf{v}_i^{t+1} = \chi\left[\mathbf{v}_i^t + c_1\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i^t) + c_2\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i^t)\right], \tag{23}$$

$$\chi = \frac{2}{\left|2 - \varphi - \sqrt{\varphi^2 - 4\varphi}\right|}, \qquad \varphi = c_1 + c_2 > 4. \tag{24}$$

With $c_1 = c_2 = 2.05$, this gives $\chi \approx 0.7298$.

6.3 Binary PSO

In the discrete binary version (Kennedy and Eberhart, 1997), the velocity update is unchanged. Each bit is then resampled from the sigmoid of its velocity:

$$s_{ij}^{t+1} = \begin{cases} 1 & \text{if } r < \sigma\left(v_{ij}^{t+1}\right), \\ 0 & \text{otherwise,} \end{cases} \qquad \sigma(v) = \frac{1}{1 + e^{-v}}. \tag{25}$$

Velocities are usually clamped to $[-V_{\max}, V_{\max}]$ (for example $V_{\max} = 4$) so that bits do not get stuck at 0 or 1.

6.4 Applications in machine learning

1. **Hyperparameter optimisation.** This is the most common use of PSO in ML. Typical targets are SVM (C, γ), random forest depth and number of trees, and learning rate, regularisation and depth for gradient boosting.

2. **Feature selection.** Use binary PSO with $F(\mathbf{s})$, for example selecting TF-IDF features for text classification.

3. **Clustering.** Each particle encodes K centroids and minimises quantisation error (van der Merwe and Engelbrecht, 2003). PSO can also seed k-means.
4. **Neural network weight training.** Train small networks without backpropagation, or use PSO to find a good starting point that gradient descent then refines.
5. **Neural architecture search.** Particles encode the number of layers, filters and kernel sizes of a convolutional network, with integer rounding.
6. **Ensemble weighting.** Particles encode the weights $\mathbf{w} \in \Delta^M$ (the probability simplex) of M base models, maximising validation performance of the weighted average.
7. **Time-series forecasting.** Tune the look-back window, the number of LSTM units and the learning rate of a forecaster.
8. **Reinforcement learning.** Search policy parameters directly (direct policy search), treating episode return as the objective.
9. **Banking: card-fraud detection.** A bank could use PSO to weight several fraud models (for example gradient boosting, a neural network and rule-based scores) so that the combined score catches as much fraud as possible at a fixed false-alarm rate. Fewer false alarms means fewer genuine customers having their cards blocked.
10. **Retail and marketing: customer segmentation.** A retailer could use PSO-based clustering to group customers by spending patterns, with each particle encoding a set of cluster centres. Searching globally avoids the poor groupings that k-means can produce from a bad start, giving marketing teams more useful segments.

6.5 Python

```
import pyswarms as ps                    # ps.single.GlobalBestPSO, ps.discrete.
    BinaryPSO
from mealpy import FloatVar, PSO         # PSO.OriginalPSO (Mealpy 3.x)
```

7 Shuffled Frog Leaping Algorithm (SFLA)

7.1 Biological inspiration

The metaphor is a population of frogs leaping between stones in a pond in search of the location with the most food. Frogs are organised into groups that share information internally, and groups periodically mix. Eusuff and Lansey (2003) formalised this as a *memetic* algorithm that combines local search within groups with global information exchange (see also Eusuff et al., 2006).

7.2 Mathematical formulation

Partitioning. The N frogs are sorted by fitness, best first, and dealt into m memeplexes in turn: frog k (in sorted order) joins memeplex $((k-1) \bmod m) + 1$. Each memeplex therefore contains $n = N/m$ frogs of mixed quality.

Local search. Within each memeplex, only the worst frog $\mathbf{X}_w$ moves. It leaps toward the memeplex's best frog $\mathbf{X}_b$:

$$\mathbf{D} = r\,(\mathbf{X}_b - \mathbf{X}_w)\,, \qquad r \sim U(0,1), \tag{26}$$

$$\mathbf{X}_w^{\text{new}} = \mathbf{X}_w + \mathbf{D}, \qquad -\mathbf{D}_{\max} \le \mathbf{D} \le \mathbf{D}_{\max}. \tag{27}$$

If $\mathbf{X}_w^{\text{new}}$ is not better than $\mathbf{X}_w$, the leap is repeated with the global best frog $\mathbf{X}_g$ in place of $\mathbf{X}_b$. If that also fails, $\mathbf{X}_w$ is replaced by a randomly generated frog (the *censorship* step). This is repeated L times per memeplex.

Shuffling. All frogs are then pooled, re-sorted and redistributed into new memeplexes. The process continues until a stopping criterion is met.

Symbol	Definition
N	Total number of frogs (population size)
m	Number of memeplexes (groups)
$n = N/m$	Number of frogs per memeplex
$\mathbf{X}_b, \mathbf{X}_w$	Best and worst frogs in the current memeplex
$\mathbf{X}_g$	Best frog in the whole population
$\mathbf{D}$	Leap vector: the change in position of the worst frog
$\mathbf{D}_{\max}$	Maximum leap allowed in each dimension
r	A uniform random number in $[0, 1]$
L	Number of local evolution steps per memeplex between shuffles

The memeplex structure is SFLA's distinguishing feature. Independent local searches preserve diversity, and shuffling spreads the best information across the population.

7.3 Binary variant

As with FA and PSO, the leap vector $\mathbf{D}$ is passed through a transfer function (Section 2.4) to update each bit of a binary solution. This yields binary SFLA for feature selection.

7.4 Applications in machine learning

1. **Hyperparameter tuning.** The memeplexes act as parallel, semi-independent searches, which suits configuration spaces with several promising regions.
2. **Feature selection.** Use binary SFLA with $F(\mathbf{s})$, particularly for high-dimensional problems where diversity matters.
3. **Clustering.** Use centroid encoding as in Section 2.5.
4. **Neural network training.** Use SFLA to optimise the weights of small networks or as a global initialiser before gradient-based fine-tuning.
5. **Ensemble and model selection.** Each frog encodes which base models to include and how to weight them.
6. **Distributed and federated search.** The memeplex structure maps naturally onto parallel workers. Each worker evolves one memeplex, and shuffling becomes a periodic synchronisation step.
7. **Traffic forecasting.** Tune the hyperparameters and look-back windows of models trained on road-sensor data.
8. **Banking: explainable credit scoring.** A lender could use SFLA to choose a small set of applicant features and tune the scoring model at the same time, excluding protected attributes such as sex and marital status. The result is a default-prediction model that stays accurate but is simple enough to explain to customers and regulators.
9. **Manufacturing: predictive maintenance.** A plant could use SFLA to select which of hundreds of machine sensors a failure-prediction model needs, cutting sensor and data costs while still warning engineers before breakdowns.

7.5 Python

No maintained SFLA implementation exists in Mealpy 3.0.3, NiaPy or PySwarms. The algorithm is short enough to implement directly with NumPy:

```python
import numpy as np

def sfla(f, lb, ub, n_frogs=30, n_memeplexes=5, n_local=10, n_shuffles=50, d_max=
    None, seed=0):
    """Minimise f over the box [lb, ub] with the Shuffled Frog Leaping Algorithm."""
    rng = np.random.default_rng(seed)
    lb, ub = np.asarray(lb, float), np.asarray(ub, float)
    d_max = (ub - lb) if d_max is None else d_max
    X = rng.uniform(lb, ub, (n_frogs, lb.size))
    F = np.array([f(x) for x in X])
    for _ in range(n_shuffles):
        order = np.argsort(F)                                  # shuffle: sort, best
    first
        X, F = X[order], F[order]
        g = X[0].copy()                                        # global best frog
        for m in range(n_memeplexes):
            idx = np.arange(m, n_frogs, n_memeplexes)          # frog k -> memeplex k
    mod m
            for _ in range(n_local):
                b, w = idx[np.argmin(F[idx])], idx[np.argmax(F[idx])]
                for target in (X[b], g):                       # leap toward local, then
     global best
                    D = np.clip(rng.random() * (target - X[w]), -d_max, d_max)
                    cand = np.clip(X[w] + D, lb, ub)
                    fc = f(cand)
                    if fc < F[w]:
                        X[w], F[w] = cand, fc
                        break
                else:                                          # censorship: random new
    frog
                    X[w] = rng.uniform(lb, ub)
                    F[w] = f(X[w])
    best = np.argmin(F)
    return X[best], F[best]

# Example with the shared objective from Section 2.6:
# best_x, best_err = sfla(objective, lb=[-3, -5], ub=[3, 1])
```

8 Comparative Summary

Table 1 compares the five algorithms.

9 Experimental Protocol for ML Studies

Comparing optimisers on ML problems is easy to do badly. We recommend the following protocol.

1. **Equal evaluation budgets.** Compare algorithms at the same number of objective evaluations, $N_{\mathrm{FE}} = N \times T$, not the same number of iterations. Each evaluation trains a model, so evaluations, not iterations, are the true cost.

Table 1: Comparison of the five metaheuristics.

	FA	**BA**	**ACO**	**PSO**	**SFLA**
Native search space	Continuous	Continuous	Discrete (graph); continuous via $\text{ACO}_{\mathbb{R}}$	Continuous	Continuous
Main mechanism	Distance-dependent attraction	Frequency-tuned velocity, loudness and pulse rate	Pheromone reinforcement and evaporation	Personal and social best attraction	Memeplex local search plus shuffling
Key parameters	β_0, γ, α	$f_{\min}$, $f_{\max}$, A^0, r^0, α, γ	α, β, ρ, m	w, c_1, c_2	N, m, L, $\mathbf{D}_{\max}$
Strength	Multimodal landscapes; finds several optima	Automatic shift from exploration to exploitation	Sequential and structural construction	Simple, fast, well studied	Preserves diversity; parallel-friendly
ML sweet spot	Diverse ensembles, HPO	Expensive HPO, feature selection	Feature selection, rule induction, NAS, pipelines	HPO, ensemble weights, clustering	Parallel HPO, feature selection
Binary variant	Transfer function	BBA (Nakamura et al., 2012)	Native	Binary PSO (Kennedy and Eberhart, 1997)	Transfer function

2. **Strong baselines.** Always include random search (Bergstra and Bengio, 2012) and Bayesian optimisation, for example via Optuna (Akiba et al., 2019) or Gaussian-process methods (Snoek et al., 2012). A metaheuristic that does not beat random search at equal budget adds complexity for nothing.

3. **Nested cross-validation.** The inner loop is used by the optimiser and the outer loop estimates generalisation. Reporting the optimiser's own best validation score overstates performance, because the search overfits the validation folds.

4. **Multiple seeds.** Run each optimiser with at least 10–30 random seeds and report the mean, standard deviation and convergence curves.

5. **Statistical testing.** For comparisons across datasets, use the Wilcoxon signed-rank test for two methods, or the Friedman test with a post-hoc Nemenyi test for several (Demšar, 2006).

6. **Report the search space.** List the bounds, scales (linear or logarithmic) and encodings of every hyperparameter, and every algorithm parameter used.

10 Limitations

1. **No free lunch.** Averaged over all possible problems, no optimiser outperforms any other (Wolpert and Macready, 1997). The choice must follow the structure of the problem.

2. **Metaphor over mechanism.** Many "novel" metaheuristics are existing techniques with a new animal metaphor (Sörensen, 2015). Algorithms should be judged by their search mechanism and by rigorous benchmarking.

3. **Sample efficiency.** Population-based methods typically need more evaluations than model-based methods such as Bayesian optimisation. This matters when each evaluation trains a deep network.

4. **Parameter sensitivity.** Each metaheuristic has its own parameters, so tuning the tuner can become a problem in its own right.

5. **No optimality guarantees.** The returned solution is good, not provably optimal. Convergence proofs exist only under restrictive assumptions.

6. **Scalability.** Weight training with metaheuristics becomes impractical as the number of weights grows. Gradient methods remain superior for large networks.

11 Conclusion

Fireflies, bats, ants, birds and frogs each embody a different strategy for searching a large space without central control. Translated into algorithms, these strategies give ML practitioners useful tools for the problems gradient descent cannot reach: hyperparameter optimisation, feature selection, architecture and pipeline search, clustering and ensemble construction. Their value is greatest when the objective is non-differentiable, the search space is mixed or discrete, and the landscape has many good regions. Used with an equal-budget, multi-seed protocol and honest baselines, these methods are a sound complement to the modern ML toolkit.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631.

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research, 13*, 281–305.

Byla, E., & Pang, W. (2019). DeepSwarm: Optimising convolutional neural networks using swarm intelligence. In *Advances in Computational Intelligence Systems (UKCI 2019)*. arXiv:1905.07350.

Clerc, M., & Kennedy, J. (2002). The particle swarm: Explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation, 6*(1), 58–73.

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research, 7*, 1–30.

Dorigo, M. (1992). *Optimization, learning and natural algorithms* [Doctoral dissertation]. Politecnico di Milano.

Dorigo, M., & Gambardella, L. M. (1997). Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation, 1*(1), 53–66.

Eusuff, M. M., & Lansey, K. E. (2003). Optimization of water distribution network design using the shuffled frog leaping algorithm. *Journal of Water Resources Planning and Management, 129*(3), 210–225.

Eusuff, M., Lansey, K., & Pasha, F. (2006). Shuffled frog-leaping algorithm: A memetic meta-heuristic for discrete optimization. *Engineering Optimization, 38*(2), 129–154.

Goss, S., Aron, S., Deneubourg, J. L., & Pasteels, J. M. (1989). Self-organized shortcuts in the Argentine ant. *Naturwissenschaften, 76*(12), 579–581.

Kapur, J. N., Sahoo, P. K., & Wong, A. K. C. (1985). A new method for gray-level picture thresholding using the entropy of the histogram. *Computer Vision, Graphics, and Image Processing, 29*(3), 273–285.

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of the IEEE International Conference on Neural Networks*, Vol. 4, pp. 1942–1948.

Kennedy, J., & Eberhart, R. C. (1997). A discrete binary version of the particle swarm algorithm. In *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics*, Vol. 5, pp. 4104–4108.

Mirjalili, S., & Lewis, A. (2013). S-shaped versus V-shaped transfer functions for binary particle swarm optimization. *Swarm and Evolutionary Computation, 9*, 1–14.

Miranda, L. J. V. (2018). PySwarms: A research toolkit for particle swarm optimization in Python. *Journal of Open Source Software, 3*(21), 433.

Nakamura, R. Y. M., Pereira, L. A. M., Costa, K. A., Rodrigues, D., Papa, J. P., & Yang, X.-S. (2012). BBA: A binary bat algorithm for feature selection. In *Proceedings of the 25th SIBGRAPI Conference on Graphics, Patterns and Images*, pp. 291–297.

Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics, 9*(1), 62–66.

Parpinelli, R. S., Lopes, H. S., & Freitas, A. A. (2002). Data mining with an ant colony optimization algorithm. *IEEE Transactions on Evolutionary Computation, 6*(4), 321–332.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Reynolds, C. W. (1987). Flocks, herds and schools: A distributed behavioral model. *ACM SIGGRAPH Computer Graphics, 21*(4), 25–34.

Shi, Y., & Eberhart, R. (1998). A modified particle swarm optimizer. In *Proceedings of the IEEE International Conference on Evolutionary Computation*, pp. 69–73.

Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems 25*, pp. 2951–2959.

Socha, K., & Dorigo, M. (2008). Ant colony optimization for continuous domains. *European Journal of Operational Research, 185*(3), 1155–1173.

Sörensen, K. (2015). Metaheuristics—the metaphor exposed. *International Transactions in Operational Research, 22*(1), 3–18.

van der Merwe, D. W., & Engelbrecht, A. P. (2003). Data clustering using particle swarm optimization. In *Proceedings of the 2003 Congress on Evolutionary Computation (CEC '03)*, Vol. 1, pp. 215–220.

Van Thieu, N., & Mirjalili, S. (2023). MEALPY: An open-source library for latest meta-heuristic algorithms in Python. *Journal of Systems Architecture, 139*, 102871.

Vrbančič, G., Brezočnik, L., Mlakar, U., Fister, D., & Fister Jr., I. (2018). NiaPy: Python microframework for building nature-inspired algorithms. *Journal of Open Source Software, 3*(23), 613.

Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation, 1*(1), 67–82.

Yang, X.-S. (2008). *Nature-inspired metaheuristic algorithms*. Luniver Press.

Yang, X.-S. (2009). Firefly algorithms for multimodal optimization. In *Stochastic Algorithms: Foundations and Applications (SAGA 2009)*, LNCS 5792, pp. 169–178. Springer.

Yang, X.-S. (2010). A new metaheuristic bat-inspired algorithm. In *Nature Inspired Cooperative Strategies for Optimization (NICSO 2010)*, Studies in Computational Intelligence 284, pp. 65–74. Springer.